Reflexión técnica: IA como herramienta, no como autor

Proyecto: LogSentinel – Python Log Analysis Tool
Actividad: Semana 3 – IA-first mindset

1. Desarrollo del proyecto y uso de inteligencia artificial

Para esta actividad desarrollé LogSentinel, un programa en Python que analiza archivos de registros de eventos en formato .txt. Su objetivo es identificar niveles de severidad, validar fechas, detectar registros incompletos y presentar un resumen de los resultados en la consola.

Durante el desarrollo utilicé ChatGPT como herramienta de apoyo para organizar el código, comprender las funciones necesarias y proponer soluciones técnicas. Sin embargo, antes de implementar el programa, definí las reglas que debía seguir, especialmente para manejar registros incorrectos.

Decidí que solamente se aceptarían las severidades INFO, WARNING y ERROR, y que las fechas deberían utilizar el formato AAAA-MM-DD. También establecí que una fecha ausente o inválida debía generar una advertencia sin descartar automáticamente el evento. En cambio, una línea vacía, una severidad desconocida o un mensaje incompleto debían considerarse registros mal formados.

Estas decisiones fueron importantes porque determinaron el comportamiento del programa y permitieron evaluar si las soluciones sugeridas por la IA cumplían realmente con los requisitos.

2. Validación y pruebas realizadas

Después de implementar el código, realicé pruebas desde Visual Studio Code utilizando tres archivos diferentes.

El primero, logs_correctos.txt, contenía cinco registros válidos y permitió comprobar que el programa identificaba correctamente los niveles de severidad sin generar advertencias innecesarias.

El segundo, logs_errores.txt, incluía ocho líneas con distintos problemas, como fechas inexistentes, severidades no reconocidas, mensajes incompletos y una línea vacía. El programa identificó tres líneas mal formadas o incompletas y tres advertencias de fecha, coincidiendo con los resultados esperados.

Finalmente, utilicé logs_edge_cases.txt para comprobar situaciones especiales, incluyendo fechas de años bisiestos y registros sin mensaje. Esta prueba permitió confirmar que el programa reconocía correctamente el 29 de febrero de 2024 como fecha válida y el 29 de febrero de 2025 como inválida.

Estas pruebas me ayudaron a comprender que un programa no debe considerarse correcto únicamente porque se ejecuta sin errores. También es necesario verificar que sus resultados coincidan con las reglas establecidas.

3. Limitación identificada y mejora de la solución de IA

Durante la revisión identifiqué una limitación en la primera versión del código sugerido por ChatGPT. Aunque los conteos eran correctos, el programa utilizaba el mismo mensaje para indicar que una fecha estaba ausente o era inválida.

Por ejemplo, mostraba la advertencia «Fecha ausente o invalida», sin explicar cuál de las dos situaciones había ocurrido.

Consideré que esta información era insuficiente para una herramienta de análisis de registros, ya que conocer la causa específica del problema facilita su revisión.

Por ese motivo, solicité una mejora para distinguir ambas situaciones. La función de análisis fue modificada para clasificar las fechas como válidas, inválidas o ausentes. Además, cuando se detecta una fecha incorrecta, el programa muestra el valor que produjo la advertencia.

Después de aplicar el cambio, repetí las pruebas para comprobar que los conteos originales no se vieran afectados. Los resultados continuaron coincidiendo con lo esperado y las advertencias se volvieron más precisas.

Esta experiencia demuestra que una respuesta generada por inteligencia artificial puede funcionar y, aun así, necesitar mejoras. La revisión humana fue necesaria para identificar una limitación de claridad y decidir cómo corregirla.

4. Aprendizajes y conclusión

El principal aprendizaje de esta actividad fue comprender que utilizar inteligencia artificial para programar no significa delegarle todas las decisiones.

ChatGPT facilitó la estructura inicial del proyecto, la explicación de conceptos y la implementación de mejoras. Sin embargo, fue necesario definir criterios propios, probar diferentes escenarios, interpretar resultados y comprobar que los cambios no introdujeran problemas nuevos.

También reforcé mis conocimientos sobre funciones, expresiones regulares, validación de fechas, manejo de archivos y estructuras de control en Python. Aprendí la importancia de utilizar archivos de prueba con datos correctos e incorrectos para evaluar el comportamiento de un programa.

Como parte del proceso, documenté el proyecto mediante un archivo README y publiqué el código en GitHub para conservarlo y facilitar su consulta.

Concluyo que la inteligencia artificial es una herramienta valiosa para acelerar el aprendizaje y el desarrollo de software, siempre que sus propuestas se revisen críticamente. El valor del trabajo no está solamente en generar código, sino en comprenderlo, establecer sus reglas, validar sus resultados y asumir la responsabilidad por la solución final.